

Experimental Evaluation of an IPsec-Based VPN

Clarisse Carvalho, Diogo Salazar, Tiago Pires

Department of Computer Science

Faculty of Sciences, University of Porto

Porto, Portugal

Abstract—This report presents the implementation and evaluation of a secure VPN using IPsec over an untrusted network. Several network configurations were considered and evaluated based on metrics such as latency and throughput. The results provide insights into the impact of IPsec on network performance and its effectiveness in ensuring secure communication.

Index Terms—IPsec, VPN, Network Security, Performance Evaluation, Secure Communication

LIST OF ABBREVIATIONS

3DES	Triple Data Encryption Standard
AES	Advanced Encryption Standard
AH	Authentication Header
CA	Certificate Authority
CBC	Cipher Block Chaining
CPU	Central Processing Unit
DF	Don't Fragment
DES	Data Encryption Standard
DH	Diffie-Hellman
EAP	Extensible Authentication Protocol
ESP	Encapsulating Security Payload
GNS3	Graphical Network Simulator 3
HMAC	Hash-based Message Authentication Code
ICMP	Internet Control Message Protocol
ICV	Integrity Check Value
IETF	Internet Engineering Task Force
IKE	Internet Key Exchange
IP	Internet Protocol
IPsec	Internet Protocol Security
ISAKMP	Internet Security Association and Key Management Protocol
ISP	Internet Service Provider
IV	Initialisation Vector
LAN	Local Area Network
MAC	Message Authentication Code
MODP	Modular Exponential (Diffie-Hellman group type)
MTU	Maximum Transmission Unit
NAT	Network Address Translation
PKI	Public Key Infrastructure
PSK	Pre-Shared Key
RSA	Rivest-Shamir-Adleman
RTT	Round-Trip Time
SA	Security Association
SHA	Secure Hash Algorithm
SPI	Security Parameters Index
TCP	Transmission Control Protocol
TLS	Transport Layer Security
UDP	User Datagram Protocol
VPN	Virtual Private Network
WAN	Wide Area Network
XAUTH	Extended Authentication
XFRM	Linux Kernel IPsec Transform Framework

I. INTRODUCTION

In today's highly connected digital landscape, the internet plays a fundamental role in facilitating communication, business operations, and access to information. However, there is

an increasing dependence on open networks, which introduces major security risks, as data transmitted online is susceptible to interception and unauthorized access. This makes secure communication essential for individuals and organizations alike.

To address these concerns, VPNs have emerged as a widely adopted solution for ensuring privacy and data integrity. VPNs establish secure communication channels over public networks by encrypting transmitted data and authenticating the parties involved. Among the available VPN technologies, IPsec is a standardized framework operating at the network layer. It offers confidentiality, data integrity, authentication and anti-replay protection.

This report details the implementation and evaluation of an IPsec-based VPN in a controlled environment. It begins with an introduction to the fundamental concepts of VPNs and IPsec, followed by a description of the project's objectives and the experimental setup. Next, a preliminary network topology is introduced to support the implementation and testing phases. Finally, it presents the evaluation methodology and performance metrics used to assess the impact of IPsec on network communication.

II. VIRTUAL PRIVATE NETWORKS (VPNS)

A. Overview

A VPN is a technology that enables secure communication over public or untrusted networks by establishing an encrypted connection between devices [1]. It allows users and networks to exchange data as though they were connected directly to a private network, even when using shared infrastructure such as the internet.

Unlike traditional physical networks, VPNs use software mechanisms to establish virtual connections rather than dedicated hardware links [2]. This flexible and scalable approach is cost-efficient and offers the same level of functionality and security as private networks.

B. Security Goals

The primary function of a VPN is to enhance the security and privacy of data transmitted across networks. As public networks are inherently insecure, VPNs employ encryption and authentication mechanisms to safeguard sensitive information from unauthorized access [1].

Encryption transforms plaintext into ciphertext using cryptographic algorithms. This ensures that, even if data is intercepted, it cannot be understood without the correct decryption

keys. Authentication mechanisms verify the identity of users and devices, preventing unauthorized entities from accessing the network. Furthermore, VPNs protect the integrity of data by detecting any modifications during transmission.

In summary, VPNs provide three key security guarantees:

- **Confidentiality:** protecting data from eavesdropping
- **Integrity:** ensuring data is not altered
- **Authentication:** verifying the identity of communicating parties

C. Tunneling Mechanisms

A VPN establishes a secure virtual connection over an untrusted network, allowing data to be transmitted safely between endpoints via public infrastructure such as the internet [2]. By isolating transmitted data within this tunnel, VPNs ensure that sensitive information remains confidential and protected from interception.

The creation of a VPN tunnel relies on several fundamental processes [2]:

- **Encapsulation:** wrapping the original data packet inside a new packet with routing information, allowing secure transmission over public networks.
- **Encryption:** converting data into an unreadable format to protect it from unauthorized access during transmission.
- **Authentication:** verifying the identity of users or devices to ensure that only trusted entities can communicate through the VPN.
- **Tunneling Protocols:** protocols (e.g., PPTP, L2TP, OpenVPN) that define how data is encapsulated, secured, and transmitted through the VPN tunnel.

Although VPN tunneling offers many advantages, the encryption and decryption processes can impact performance. These processes can increase latency and reduce throughput.

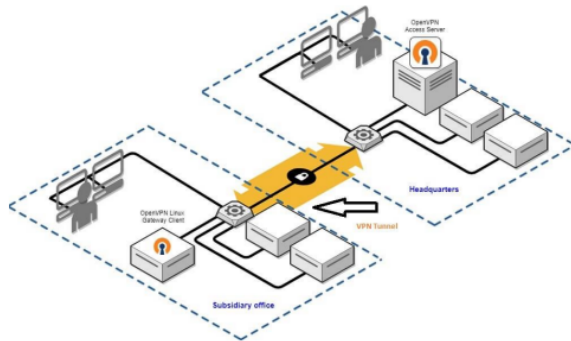


Fig. 1. VPN Tunnel Architecture [2]

D. Deployment Models

A remote access VPN allows users to securely connect to a private network from a distance [3]. This approach is particularly useful for remote working or for accessing the network on the go, as it allows users to interact with internal resources as though they were physically connected to the organization's network.

In this model, users install a VPN client on their devices to establish a connection. They are then authenticated by a VPN server using the appropriate mechanisms. Once this process is complete, a secure tunnel is created between the user's device and the organization's network. This ensures that all transmitted data is protected by encryption and secure communication protocols.

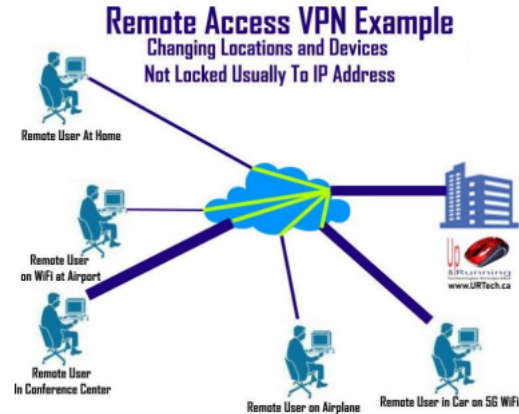


Fig. 2. Remote Access VPN Example [2]

A site-to-site VPN connects entire networks in different geographical areas, such as branch offices and headquarters. Rather than individual users connecting to the VPN, network gateways (such as routers or firewalls) establish the secure connection.

In this configuration, VPN gateways are deployed at each site to establish and maintain secure tunnels between networks. As a result, communication between devices across the connected networks is transparent and does not require direct user intervention. Furthermore, end devices do not require additional VPN client software since the network infrastructure handles the encryption and tunneling processes entirely.

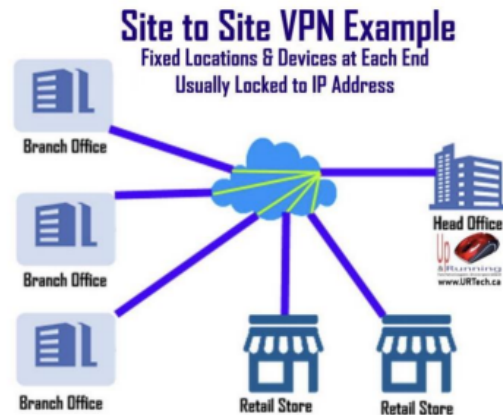


Fig. 3. Site-to-Site VPN [2]

III. IPSEC

A. Overview

IPsec is a set of open standards developed by the IETF to enable secure and private communication over IP networks. IPsec enables systems to select the necessary security protocols and determine which algorithm(s) to use for each service. It also establishes the cryptographic keys required for these services [3].

IPsec operates at the network layer, enabling it to encrypt and authenticate IP traffic independently of the application layer. Although this allows IPsec to transparently protect a wide range of protocols, it does not replace application-layer security mechanisms, such as TLS. Both can coexist: TLS secures the application payload end-to-end between hosts, whereas IPsec secures the network path between endpoints or gateways. However, these may not align with the applications communicating with each other.

The IPsec architecture is based on three core principles: authentication, confidentiality, and key management. These principles are realized through protocols such as the AH and, most notably, the ESP. These protocols provide the following security services [3]:

- **Confidentiality:** enables communicating nodes to encrypt messages to prevent eavesdropping by third parties
- **Data Integrity:** ensures that the received packet has not been altered or tampered with while in transit
- **Authentication:** known as data origin authentication, this service verifies that the packet was actually transmitted by the party identified as the source in the header
- **Anti-replay Protection:** this service thwarts replay attacks, where an attacker captures an authenticated packet and retransmits it later, by using sequence numbers to identify and reject duplicate packets

B. AH and ESP Protocols

The AH is an IPsec protocol that ensures data integrity, verifies the source of IP packets and prevents replay attacks. Its main purpose is to verify that transmitted data has not been altered and originates from a trusted source [4].

AH works by adding a header containing an ICV. This ICV is a cryptographic hash generated using a shared secret key. The receiver then recalculates this value to verify the packet's integrity and authenticity. Sequence numbers are also used to prevent replay attacks.

In practice, AH is rarely used in modern deployments due to its incompatibility with NAT. The ICV covers parts of the IP header that are modified by NAT, which results in failed authentication. For this reason, ESP is the standard choice, and AH was not used in any of the project's configurations.

The AH header includes key fields such as the SPI and the sequence number, as well as the authentication data. AH is identified in IP packets by protocol number 51.

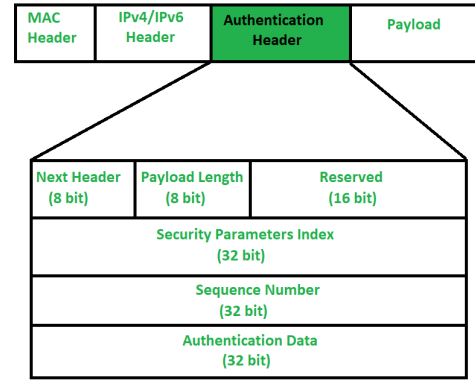


Fig. 4. IPsec AH Format

The ESP is an IPsec protocol that provides data confidentiality and, if desired, data integrity, authentication and protection against replay attacks. Its primary function is to protect the contents of IP packets by encrypting transmitted data [5].

ESP works by encapsulating and encrypting the packet's payload. Depending on the configuration, it may also include an ICV to verify the authenticity of the data. Sequence numbers are used to prevent replay attacks.

Unlike AH, ESP provides encryption, which is why it is the most widely deployed protocol for IPsec VPNs. It can operate in transport or tunnel mode.

The ESP header includes key fields such as the SPI and the sequence number, as well as the payload data and the optional ICV.

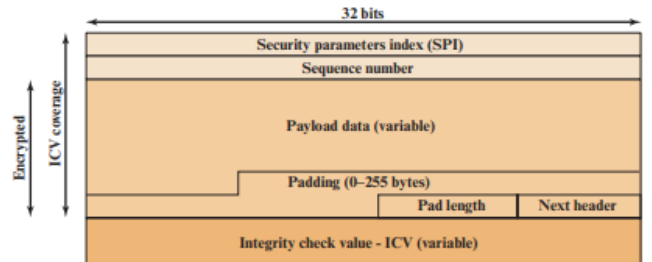


Fig. 5. IPsec ESP Format

C. Modes of Operation

IPsec supports two operating modes: transport mode and tunnel mode. These determine how data is protected during transmission.

In transport mode, only the payload of the upper layer is protected, while the original IP header is preserved. When ESP is used, the payload is encrypted and can be optionally authenticated. When AH is used, selected fields of the IP header and the full payload are authenticated, but no encryption is applied. Since the original IP header remains visible, transport mode does not conceal the identity of the communicating endpoints. This mode is suited to end-to-end communication between two hosts that both implement IPsec directly, such as a remote client connecting to a server [3].

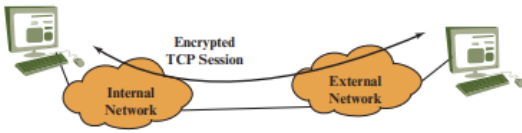


Fig. 6. IPsec Transport Mode [3]

In contrast, tunnel mode protects the entire IP packet, including both the header and the payload. The original packet is encapsulated within a new IP packet containing a new outer header, effectively creating a secure 'tunnel' between the endpoints. This prevents intermediate network devices from inspecting the original packet, thereby enhancing security. Using new source and destination addresses in the outer header also conceals the internal network structure.

Tunnel mode is commonly used when one or both endpoints are security gateways, such as routers or firewalls. It enables multiple devices within a network to communicate securely without IPsec needing to be implemented on each individual host. For this reason, tunnel mode is widely used in site-to-site VPNs, where entire networks are securely interconnected over public infrastructures such as the internet.

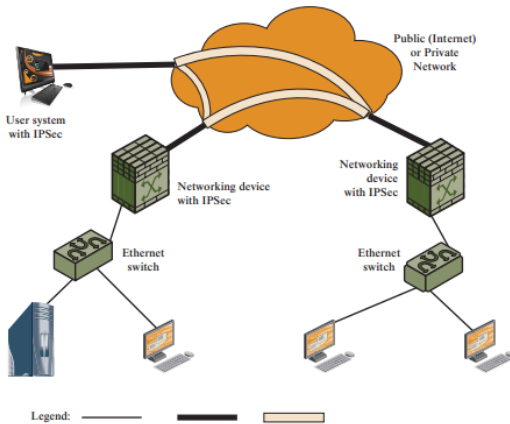


Fig. 7. IPsec Tunnel Mode [3]

D. Security Associations (SAs)

A fundamental concept in IPsec is the SA, which underpins the authentication and confidentiality mechanisms. An SA is a unidirectional, logical connection between a sender and receiver that secures transmitted traffic. Therefore, secure, bidirectional communication requires two separate SAs, one for each direction.

Each SA is uniquely identified by a set of parameters that enable the receiving system to correctly process incoming packets [3]:

- **Security Parameters Index (SPI):** A 32-bit identifier assigned to the SA, with local significance. The SPI is included in the AH or ESP header and is used by the receiver to determine which SA should be applied to the packet.

- **IP Destination Address:** Specifies the endpoint of the SA, which may correspond to a host or a network device such as a router or firewall.
- **Security Protocol Identifier:** Indicates whether the SA is associated with the AH or ESP protocol.

In practice, a security association is identified by the combination of the destination IP address and the SPI value within the IPsec header. This mechanism allows secure communication channels to be recognized and managed efficiently within IPsec implementations.

E. Anti-Replay Protection

IPsec incorporates an anti-replay mechanism to protect against replay attacks. In these attacks, an attacker intercepts a valid packet and retransmits it in order to either disrupt communication or perform unauthorized actions.

This protection is achieved using the Sequence Number field, which is associated with each SA. At the start of an SA, the sender initializes the sequence number and increases it for each packet transmitted. To prevent reuse, the sequence number must not wrap around; once the maximum value is reached, a new SA must be established.

At the receiving end, a sliding window mechanism is used to track valid packets and identify duplicates. Packets that fall within the acceptable window and have not been received before are authenticated and accepted. Duplicates, packets outside the window and packets that fail authentication are discarded.

This process ensures that only fresh, valid packets are processed, which effectively prevents replay-based attacks.

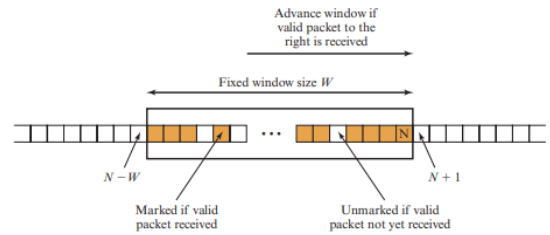


Fig. 8. IPsec Anti-Replay Sliding Window Mechanism [3]

F. IPsec Performance Trade-offs

The implementation of IPsec within a network architecture necessitates a careful balance between the enforcement of robust security policies and the maintenance of network efficiency. This inherent trade-off is primarily characterized by the computational burden imposed on the processing units and the subsequent impact on transmission metrics.

The cryptographic operations required for encryption and integrity verification, such as those performed by the AES and SHA algorithms, demand significant CPU cycles at both the source and destination nodes. This processing overhead introduces a measurable increase in end-to-end latency and can potentially become a bottleneck that limits the maximum achievable throughput, especially in hardware environments without dedicated cryptographic acceleration.

Beyond computational constraints, IPsec introduces structural changes to the data units that affect network resource utilization. The encapsulation process adds security-related headers and trailers to the original IP packets, which increases the total size of each transmitted unit. This expansion reduces the effective bandwidth, as a larger portion of the transmission is dedicated to security metadata rather than application data. Furthermore, the increased packet dimensions may exceed the Path MTU, leading to packet fragmentation. Such fragmentation necessitates additional reassembly procedures at the receiving end, which further degrades performance and increases the risk of packet loss in congested environments.

The magnitude of these performance penalties is also highly sensitive to the specific configuration of the IPsec stack. For instance, the use of tunnel mode provides superior security by concealing the internal IP structure, yet it incurs a higher overhead compared to transport mode due to the addition of an extra outer IP header. Similarly, the choice between the Authentication Header and the Encapsulating Security Payload protocol represents a choice between lightweight integrity protection and resource-intensive confidentiality. Ultimately, these trade-offs highlight the necessity of a tailored approach to VPN deployment, where security parameters must be optimized based on the specific performance requirements and threat models of the target environment.

IV. OBJECTIVES AND EXPERIMENTAL SETUP

This section outlines the approach adopted for the implementation, configuration and evaluation of an IPsec-based VPN in a controlled environment. It defines the project's objectives, describes the experimental environment and illustrates the planned network topologies. It also provides details of the IPsec configuration parameters and introduces the evaluation methodology.

A. Objectives

The main objectives of this project were to implement and configure a secure VPN using IPsec, validate the functionality of the established tunnels, and evaluate performance under different conditions. Key metrics such as throughput and end-to-end latency were measured to assess the impact of IPsec on network performance. The project also analyzed the effect of different configurations and identified practical limitations in realistic network scenarios.

B. Experimental Environment

Several network virtualization and simulation tools are available, including Packet Tracer, GNS3, OpenStack, and EVE-NG [6]. These tools facilitate the creation of virtual network topologies comprising routers and firewalls, as well as the configuration of secure tunnels and routing protocols. By emulating real network devices and operating systems, they provide a practical platform for validating configurations, analyzing behavior, and identifying errors, thereby eliminating the need for physical infrastructure.

Although there were several alternatives, GNS3 (Graphical Network Simulator 3) was selected due to prior experience with the platform. GNS3 is an open-source network virtualization tool that enables users to design and test real-world router and switch configurations in a virtual environment [7]. It provides a graphical interface for building topologies and configuring devices via virtual consoles, and supports the same operating systems used by real network devices.

C. Network Topology

Two distinct topologies were implemented, each corresponding to a different VPN deployment scenario. Both were realised in GNS3 with the fewest possible virtualized devices to ensure experimental control and reproducibility.

1) *Site-to-Site Topology*: This topology simulates two geographically separate networks interconnected via a public network. It consists of three devices:

- **Two gateway routers**: each configured with IPsec, responsible for establishing and maintaining the encrypted tunnel
- **One intermediate router**: emulating the public Internet between both sites

IPsec is configured in tunnel mode between the two gateways, ensuring that all inter-site traffic is encrypted.

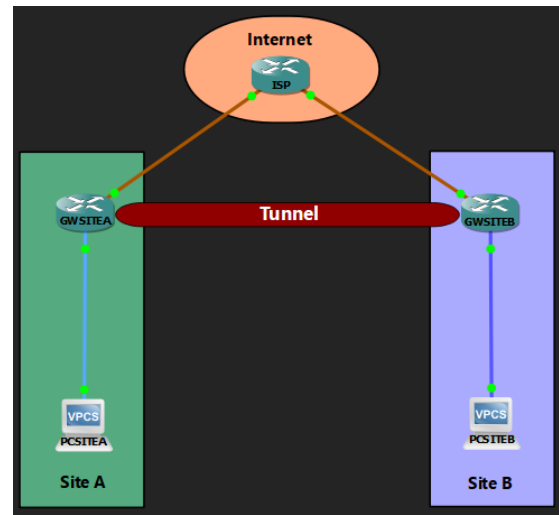


Fig. 9. Planned Site-to-Site Topology in GNS3

2) *Remote Access Topology*: This topology simulates a single remote user securely connecting to a private network via a public one. It also consists of three devices:

- **One remote host**: acting as the VPN client
- **One intermediate router**: simulating the public internet
- **One gateway router**: configured to accept IPsec remote access connections

IPsec is configured in tunnel mode between the remote host and the gateway to secure the connection at the endpoint level. This configuration is commonly used for remote working scenarios, when a user accesses internal resources from an external location.

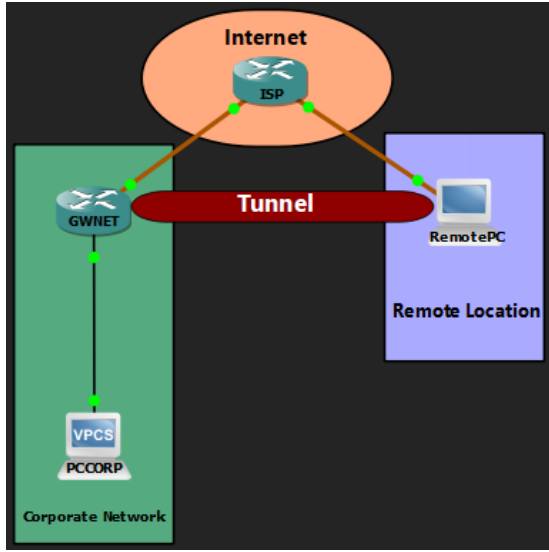


Fig. 10. Planned Remote Access Topology in GNS3

D. IPsec Configuration

The IPsec tunnels were configured using the parameters listed in Table I, which summarizes the candidate values considered during the planning phase. The parameters selected for each implemented scenario are detailed in Table V and Table VI.

TABLE I
PLANNED IPSEC CONFIGURATION PARAMETERS

Parameter	Candidate Values
Key exchange protocol	IKEv1 / IKEv2
Encryption algorithm	AES-128 / AES-256 / 3DES
Integrity / hash algorithm	SHA-1 / SHA-256
Authentication method	Pre-shared key (PSK)
Site-to-site mode	Tunnel (ESP)
Remote access mode	Tunnel (ESP)

E. Evaluation Metrics

The two primary performance metrics are throughput and end-to-end latency. Throughput is the volume of data successfully transmitted over the VPN within a given time period. Latency, on the other hand, is the time taken for packets to travel through the encrypted tunnel.

Latency was measured using ICMP echo requests (ping), recording RTT across both topologies with and without IPsec enabled. Throughput was measured using *iperf*, generating UDP and TCP traffic streams between endpoints and recording the achieved data rate. To account for variability in the virtualised environment, each experiment was repeated five times. Baseline measurements were collected with IPsec disabled, followed by measurements with IPsec enabled, enabling direct comparison of the encryption overhead

V. IMPLEMENTATION

This section describes the final implementation of the IPsec VPN solution in GNS3, including the network topologies, IPsec configuration parameters, router configuration, and tunnel validation procedures.

A. Final GNS3 Topologies

Three VPN deployment scenarios were implemented and evaluated: one site-to-site VPN and two remote access VPN.

1) *Site-to-Site VPN Topology*: On GwSiteA, interface FastEthernet0/0 was assigned 10.0.11.1/24, connecting to the SiteA LAN, and interface FastEthernet0/1 was assigned 10.0.12.1/24, connecting to the simulated Internet. A default static route was configured pointing to the ISP (10.0.12.2) as the next hop for all external traffic:

```
ip route 0.0.0.0 0.0.0.0 10.0.12.2
```

GwSiteB was configured symmetrically. Interface FastEthernet0/0 was assigned 10.0.22.1/24 for the Site B LAN, and interface FastEthernet0/1 was assigned 10.0.23.1/24 for the Internet segment. Its default route pointed to the ISP via 10.0.23.2:

```
ip route 0.0.0.0 0.0.0.0 10.0.23.2
```

The ISP router required no routing beyond its directly connected interfaces. It was configured with FastEthernet0/0 at 10.0.12.2/24 and FastEthernet1/0 at 10.0.23.2/24, providing IP connectivity between the two gateway routers.

End hosts were configured as follows: PCSiteA was assigned 10.0.11.2/24 with a default gateway of 10.0.11.1 (GwSiteA), and PCSiteB was assigned 10.0.22.2/24 with a default gateway of 10.0.22.1 (GwSiteB).

TABLE II
SITE-TO-SITE TOPOLOGY ADDRESSING SCHEME

Device	Interface	IP Address
GwSiteA	F0/0	10.0.11.1/24
GwSiteA	F0/1	10.0.12.1/24
ISP	F0/0	10.0.12.2/24
ISP	F1/0	10.0.23.2/24
GwSiteB	F0/0	10.0.22.1/24
GwSiteB	F0/1	10.0.23.1/24

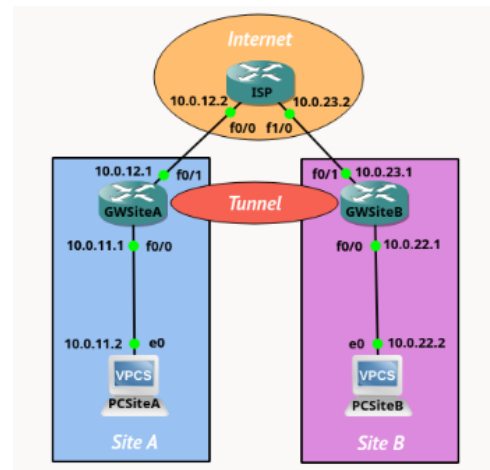


Fig. 11. Final Site-to-Site VPN Topology in GNS3

2) *Remote Access Topology with VPNC*: On GwCorp, interface FastEthernet0/0 was assigned 10.0.11.1/24, connecting to the internal corporate LAN, and interface FastEthernet0/1 was assigned 10.0.12.1/24, connecting to the simulated Internet. A default static route was configured pointing to the ISP (10.0.12.2) as the next hop for all external traffic:

```
ip route 0.0.0.0 0.0.0.0 10.0.12.2
```

The ISP router was configured identically to the site-to-site scenario, with FastEthernet0/0 at 10.0.12.2/24 facing the corporate network and FastEthernet1/0 at 10.0.23.2/24 facing the remote location. No additional routing configuration was required beyond its directly connected interfaces.

RemotePC was assigned address 10.0.23.1/24 on interface ens3, with a default gateway of 10.0.23.2 (ISP). Unlike the site-to-site scenario where IPsec is handled transparently by dedicated gateway routers, in this topology RemotePC participates directly in the IPsec negotiation, acting simultaneously as the VPN client and the traffic endpoint.

The internal host PCCorp was assigned 10.0.11.2/24 with a default gateway of 10.0.11.1 (GwCorp), representing an organizational resource accessible only through the VPN tunnel.

TABLE III
REMOTE ACCESS TOPOLOGY ADDRESSING SCHEME

Device	Interface	IP Address
RemotePC	E0	10.0.23.1/24
GwCorp	F0/0	10.0.11.1/24
GwCorp	F0/1	10.0.12.1/24
ISP	F0/0	10.0.12.2/24
ISP	F1/0	10.0.23.2/24

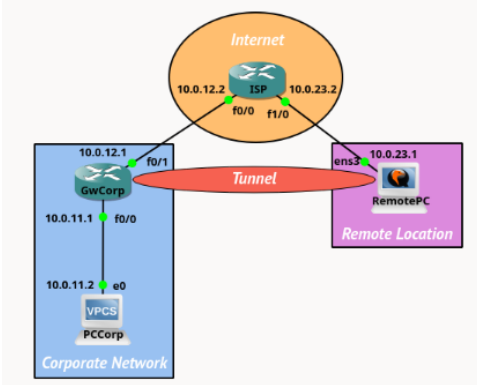


Fig. 12. Final Remote Access VPN Topology in GNS3

3) *Remote Access Topology with StrongSwan*: A second remote access topology was implemented using StrongSwan as the VPN solution, replacing the Cisco IOS gateway and the vpnc client with Linux-based hosts running StrongSwan. The network addressing and physical layout are identical to the vpnc-based remote access topology, with the exception

that the Cisco gateway router (GwCorp) was replaced by a Linux server running StrongSwan, enabling the use of IKEv2 and certificate-based authentication - features not supported by the legacy Cisco IOS configuration used in the previous scenario.

TABLE IV
REMOTE ACCESS STRONGSWAN TOPOLOGY ADDRESSING SCHEME

Device	Interface	IP Address
RemotePC	ens3	10.0.23.1/24
Server	ens3	10.0.12.1/24
Server	ens4	10.0.11.1/24
ISP	F0/0	10.0.12.2/24
ISP	F1/0	10.0.23.2/24
PCCorp	ens3	10.0.11.2/24

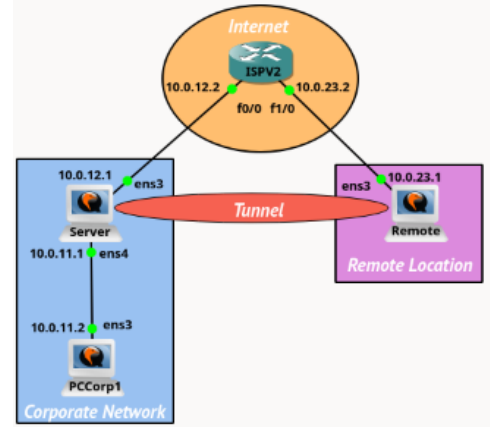


Fig. 13. Final Remote Access VPN Topology with StrongSwan in GNS3

B. IPsec Configuration - Site-to-Site

1) *Configuration Parameters*: The IPsec tunnel between GwSiteA and GwSiteB was implemented using IKEv1 for key exchange and ESP for encapsulation and encryption. The specific algorithms and authentication method selected are summarised in Table V.

TABLE V
IPSEC CONFIGURATION PARAMETERS - SITE-TO-SITE

Parameter	Selected Value
Key Exchange Protocol	IKEv1
Encryption Algorithm	AES-128
Integrity / Hash Algorithm	SHA-1 (HMAC variant)
Authentication Method	Pre-Shared Key (PSK)
Diffie-Hellman Group	Group 2 (1024-bit)
IPsec Protocol	ESP
IPsec Mode	Tunnel Mode (default)

2) *IKE Phase 1 - ISAKMP Policy*: IKE Phase 1 establishes the secure authenticated channel used to negotiate the IPsec Security Associations. On both gateway routers, a single ISAKMP policy (policy 1) was configured specifying AES-128 encryption, pre-shared key authentication, and Diffie-Hellman Group 2. The integrity algorithm defaults to SHA-1 when not explicitly stated. A pre-shared key was configured

on each gateway, referencing the other's external IP address as the peer:

GwSiteA:

```
crypto isakmp policy 1
  encr aes
  authentication pre-share
  group 2
crypto isakmp key Secret_Key_123
address 10.0.23.1
```

GwSiteB:

```
crypto isakmp policy 1
  encr aes
  authentication pre-share
  group 2
crypto isakmp key Secret_Key_123
address 10.0.12.1
```

Both gateways share the same pre-shared key (Secret_Key_123), with each router configured to authenticate against the other's external interface address - 10.0.23.1 from the perspective of GwSiteA, and 10.0.12.1 from the perspective of GwSiteB.

3) *IKE Phase 2 - Transform Set and Crypto Map:* IKE Phase 2 negotiates the IPsec Security Associations that protect the actual data traffic. A transform set named Tar_Set was defined identically on both gateways, specifying ESP with AES-128 encryption and SHA-1 HMAC for integrity verification. Since no explicit mode was configured, the Cisco IOS default of tunnel mode applies:

```
crypto ipsec transform-set Tar_Set
esp-aes esp-sha-hmac
```

A crypto map named Tar_Map_VPN was created on each gateway to bind together the remote peer address, the transform set, and the access list that defines the traffic to be protected. Access list 100 selects packets originating from the local private network and destined for the remote private network:

GwSiteA:

```
crypto map Tar_Map_VPN 1 ipsec-isakmp
  set peer 10.0.23.1
  set transform-set Tar_Set
  match address 100
access-list 100 permit ip
10.0.11.0 0.0.0.255 10.0.22.0 0.0.0.255
```

GwSiteB:

```
crypto map Tar_Map_VPN 1 ipsec-isakmp
  set peer 10.0.12.1
  set transform-set Tar_Set
  match address 100
access-list 100 permit ip
10.0.22.0 0.0.0.255 10.0.11.0 0.0.0.255
```

The crypto map was applied to the external interface (FastEthernet0/1) of each gateway, instructing the router to intercept matching outbound traffic and encapsulate it within an IPsec tunnel before forwarding it across the public network:

```
interface FastEthernet0/1
  crypto map Tar_Map_VPN
```

When a packet from PCSiteA (10.0.11.2) destined for PCSiteB (10.0.22.2) reaches GwSiteA, it matches access list 100, triggering IKE negotiation with GwSiteB if no active Security Association exists. Once the tunnel is established, the original IP packet is encapsulated within a new outer IP packet with GwSiteA's external address as source and GwSiteB's external address as destination, encrypted with AES-128, and forwarded through the ISP to the remote gateway, which decapsulates and delivers it to PCSiteB.

C. Remote Access IPsec Configuration - VPNC

The remote access topology required a fundamentally different IPsec configuration from the site-to-site scenario. Rather than a static tunnel between two known peers, the gateway was configured to accept connections from dynamic clients whose IP addresses are not known in advance. On the client side, the open-source vpnc daemon was used to establish the IPsec tunnel from the Linux-based RemotePC.

a) *Server-Side Configuration (GwCorp):* Two ISAKMP policies were defined on GwCorp to support multiple negotiation options. Policy 10 specified AES-256 encryption, SHA-256 integrity, pre-shared key authentication, and DH Group 14. Policy 20 provided a fallback using 3DES encryption and Group 2, broadening compatibility with different client configurations:

```
crypto isakmp policy 10
  encr aes 256
  hash sha256
  authentication pre-share
  group 14
crypto isakmp policy 20
  encr 3des
  authentication pre-share
  group 2
crypto isakmp key TAR_KEY_2026 address
0.0.0.0
```

The wildcard peer address (0.0.0.0) allows the gateway to accept connections from any remote IP address, which is required in dynamic remote access scenarios where the client address is not fixed.

A client configuration group named TAR_VPN_GROUP was defined to specify the parameters assigned to connecting clients, including the pre-shared key, the IP address pool from which clients receive a virtual address, and the access control list defining permitted traffic:

```
crypto isakmp client configuration
```



```
group TAR_VPN_GROUP
key TAR_KEY_2026
pool TAR_VPN_POOL
acl TAR_ACL_VPN
```

Three transform sets were defined to accommodate different combinations of encryption and integrity algorithms, all operating in tunnel mode:

```
crypto ipsec transform-set TAR_TS
esp-aes 256 esp-sha256-hmac
mode tunnel
crypto ipsec transform-set TAR_TS2
esp-des esp-sha-hmac
mode tunnel
crypto ipsec transform-set TAR_TS3
esp-des esp-md5-hmac
mode tunnel
```

A dynamic crypto map named DYNMAP was created to handle inbound connections from clients with unknown IP addresses. It referenced all three transform sets in order of preference and enabled reverse-route injection so that the gateway automatically learns routes back to connected clients:

```
crypto dynamic-map DYNMAP 10
set transform-set TAR_TS TAR_TS2 TAR_TS3
reverse-route
```

The static crypto map TAR_VPN_MAP was then configured to handle client authentication, authorisation, and address assignment, referencing the dynamic map for the actual IPsec negotiation:

```
crypto map TAR_VPN_MAP client
authentication list TAR_VPN_AUTH
crypto map TAR_VPN_MAP isakmp
authorization list TAR_VPN_AUTHZ
crypto map TAR_VPN_MAP
client configuration address respond
crypto map TAR_VPN_MAP 10 ipsec-isakmp
dynamic DYNMAP
```

An IP address pool was defined to assign virtual addresses to connecting clients in the range 10.0.100.1-10.0.100.254:

```
ip local pool TAR_VPN_POOL
10.0.100.1 10.0.100.254
```

The access list TAR_ACL_VPN defined the traffic permitted through the VPN, allowing any source from the remote client to reach the internal corporate network (10.0.11.0/24):

```
ip access-list extended TAR_ACL_VPN
permit ip 10.0.11.0 0.0.0.255 any
```

The crypto map was applied to the external interface FastEthernet0/1 (10.0.12.1/24), and a default route was configured via the ISP:

```
interface FastEthernet0/1
```

```
ip address 10.0.12.1 255.255.255.0
crypto map TAR_VPN_MAP
```

```
ip route 0.0.0.0 0.0.0.0 10.0.12.2
```

b) Client-Side Configuration (RemotePC): On RemotePC, the vpnc client was configured through the file /etc/vpnc/corporate.conf. This file specified the gateway address, the group name and shared secret matching those defined on GwCorp, and the Xauth credentials used for user-level authentication:

```
IPSec gateway 10.0.12.1
IPSec ID TAR_VPN_GROUP
IPSec secret TAR_KEY_2026
Xauth username tarteste
Xauth password tarZ026
Enable weak encryption
```

The Enable weak encryption directive was necessary to permit negotiation of the 3DES and DES transform sets defined in policies 20 and TAR_TS2/TAR_TS3 on the server side, as vpnc disables these by default due to their reduced security margin.

Upon successful negotiation, vpnc established the IPsec tunnel and assigned RemotePC a virtual IP address from the TAR_VPN_POOL pool, enabling it to communicate with internal corporate resources as though it were directly connected to the 10.0.11.0/24 network.

D. Remote Access IPsec Configuration - StrongSwan

The StrongSwan-based remote access topology used IKEv2 with EAP-MSCHAPv2 for user authentication and certificate-based server authentication, representing a more modern and secure approach compared to the IKEv1/XAUTH configuration used with vpnc.

The specific parameters selected for this implementation are summarised in Table VI.

TABLE VI
IPSEC CONFIGURATION PARAMETERS - STRONGSWAN REMOTE ACCESS

Parameter	Selected Value
Key Exchange Protocol	IKEv2
Encryption Algorithm	AES-256
Integrity / Hash Algorithm	SHA-256
Diffie-Hellman Group	MODP-2048 (Group 14)
Server Authentication	PKI Certificate (RSA 2048-bit)
User Authentication	EAP-MSCHAPv2
IPsec Protocol	ESP
IPsec Mode	Tunnel Mode
IP Pool	10.0.100.1 - 10.0.100.254

1) Server-Side Configuration (StrongSwan Gateway): On the server side, StrongSwan was configured through the swanctl.conf file. The connection named remote-access defined the local address as the WAN interface (10.0.12.1) and accepted connections from any remote address. Server authentication was performed using a PKI certificate (gwcorp-cert.pem), while client authentication used EAP-MSCHAPv2:

```

connections {
    remote-access {
        local_addrs = 10.0.12.1
        remote_addrs = %any
        local {
            auth = pubkey
            certs = gwcorp-cert.pem
            id = 10.0.12.1
        }
        remote {
            auth = eap-mschapv2
            eap_id = %any
        }
        children {
            remote-access {
                local_ts = 10.0.11.0/24
                remote_ts = dynamic
                esp_proposals =
                aes256-sha256-modp2048
                mode = tunnel
            }
        }
        version = 2
        proposals = aes256-sha256-modp2048
        pools = vpn-pool
    }
}
pools {
    vpn-pool {
        addrs = 10.0.100.1 - 10.0.100.254
        dns = 8.8.8.8
    }
}
secrets {
    eap-remoteuser {
        id = tarteste
        secret = "tar2026"
    }
}

```

IP forwarding was enabled on the server to allow traffic from VPN clients to reach the internal corporate network:

```

echo "net.ipv4.ip_forward=1"
| sudo tee -a /etc/sysctl.conf
sudo sysctl -p

```

NAT rules were configured to masquerade VPN client traffic on the internal interface:

```

sudo iptables -t nat -A POSTROUTING \
    -s 10.0.100.0/24 -o ens4 -j MASQUERADE

```

Server authentication relied on a self-signed PKI infrastructure. A root CA was generated locally and used to issue a server certificate for 10.0.12.1, which was then installed in the StrongSwan certificate store:

```

pki --gen --type rsa --size 4096

```

```

--outform pem > ca-key.pem
pki --self --ca --lifetime 3650 --in
ca-key.pem \
--dn "CN=VPN Root CA" --outform pem >
ca-cert.pem
pki --gen --type rsa --size 2048
--outform pem > gwcorp-key.pem
pki --issue --lifetime 1825 --cacert
ca-cert.pem \
--cakey ca-key.pem --in gwcorp-key.pem \
--dn "CN=10.0.12.1" --san "10.0.12.1" \
--flag serverAuth --outform pem >
gwcorp-cert.pem

```

2) *Client-Side Configuration (RemotePC):* On RemotePC, StrongSwan was also installed and configured as a client through a swanctl.conf file. The CA certificate generated on the server was copied to the client and installed in the StrongSwan certificate store to allow verification of the server's identity. The client authenticated using EAP-MSCHAPv2 credentials:

```

connections {
    vpn-cliente {
        local_addrs = 10.0.23.1
        remote_addrs = 10.0.12.1
        local {
            auth = eap
            eap_id = tarteste
        }
        remote {
            auth = pubkey
            id = 10.0.12.1
        }
        children {
            vpn-cliente {
                remote_ts = 10.0.11.0/24
                local_ts = dynamic
                esp_proposals
                = aes256-sha256-modp2048
                mode = tunnel
            }
        }
        version = 2
        proposals = aes256-sha256-modp2048
        vips = 0.0.0.0
    }
}
secrets {
    eap-remoteuser {
        id = tarteste
        secret = "tar2026"
    }
}

```

The tunnel was initiated using:

```

sudo swanctl --load-all

```

```
sudo swanctl --initiate --child
vpn-cliente
```

Upon successful negotiation, RemotePC received the virtual IP address 10.0.100.1/32 from the server pool, assigned directly to the ens3 interface via the XFRM subsystem, without creating a separate virtual tunnel interface as vpnc does.

E. Tunnel Validation

After completing the configuration process, the VPN tunnels were validated using Cisco IOS monitoring commands on both gateway routers. The following commands were used to inspect the state of the IKE and IPsec Security Associations:

```
show crypto isakmp sa
show crypto ipsec sa
```

1) *Site-to-Site Topology*: The ISAKMP SA output on both GwSiteA and GwSiteB confirmed that IKE Phase 1 negotiation completed successfully. The SA state QM_IDLE indicates that the ISAKMP SA is established and idle, meaning Phase 1 has concluded and the channel is available for Phase 2 re-keying when needed. The peer addresses (10.0.23.1 and 10.0.12.1) and the connection identifier (conn-id 1001) were consistent across both gateways, confirming bidirectional SA establishment.

```
GwSiteA#show crypto isakmp sa
IPv4 Crypto ISAKMP SA
dst      src      state      conn-id slot status
10.0.23.1 10.0.12.1 QM_IDLE    1001     0 ACTIVE
```

Fig. 14. ISAKMP SA on GwSiteA - QM_IDLE confirms Phase 1 success

```
GwSiteB#show crypto isakmp sa
IPv4 Crypto ISAKMP SA
dst      src      state      conn-id slot status
10.0.23.1 10.0.12.1 QM_IDLE    1001     0 ACTIVE
```

Fig. 15. ISAKMP SA on GwSiteB - matching SA with peer 10.0.12.1

The show crypto ipsec sa output on GwSiteA showed the crypto map Tar_Map_VPN active on interface FastEthernet0/1 (local address 10.0.12.1). Two ESP Security Associations were present, one inbound and one outbound, both using the esp-aes esp-sha-hmac transform set in tunnel mode, with status ACTIVE. The traffic selectors confirmed that the protected traffic flowed between 10.0.11.0/255.255.255.0 (Site A) and 10.0.22.0/255.255.255.0 (Site B). Packet counters showed 5 encapsulated and 4 decapsulated packets on GwSiteA, with the inverse observed on GwSiteB (4 encapsulated, 5 decapsulated), confirming that encrypted traffic was successfully transmitted and received in both directions.

```
GwSiteA#show crypto ipsec sa
Interface: FastEthernet0/1
Crypto map tag: Tar_Map_VPN, local addr 10.0.12.1

protected vrf: (none)
local ident (addr/mask/prot/port): (10.0.11.0/255.255.255.0/0/0)
remote ident (addr/mask/prot/port): (10.0.22.0/255.255.255.0/0/0)
current_peer 10.0.23.1 port 500
  PERMIT, flags={origin_is_acl,}
#pkts encaps: 4, #pkts encrypt: 4, #pkts digest: 4
#pkts decaps: 3, #pkts decrypt: 3, #pkts verify: 3
#pkts compressed: 0, #pkts decompressed: 0
#pkts not compressed: 0, #pkts compr. failed: 0
#pkts not decompressed: 0, #pkts decompress failed: 0
#send errors 1, #recv errors 0

local crypto endpt.: 10.0.12.1, remote crypto endpt.: 10.0.23.1
path mtu 1500, ip mtu 1500, ip mtu idb FastEthernet0/1
current outbound spi: 0x808C1153(2156663123)

Inbound esp sas:
  spi: 0x808C1153(2156663123)
  transform: esp-aes esp-sha-hmac ,
  in use settings ={Tunnel,}
  conn id: 1, flow_id: SW-1, crypto map: Tar_Map_VPN
  sa timing: remaining key lifetime (k/sec): (4488698/3568)
  IV size: 16 bytes
  replay detection support: Y
  Status: ACTIVE

Inbound ah sas:

Inbound pcp sas:

outbound esp sas:
  spi: 0xBEBBCD21(3199978785)
  transform: esp-aes esp-sha-hmac ,
  in use settings ={Tunnel,}
  conn id: 2, flow_id: SW-2, crypto map: Tar_Map_VPN
  sa timing: remaining key lifetime (k/sec): (4488698/3564)
  IV size: 16 bytes
  replay detection support: Y
  Status: ACTIVE

outbound ah sas:

outbound pcp sas:
```

Fig. 16. IPsec SA on GwSiteA - active inbound and outbound ESP SAs

```
GwSiteB#show crypto ipsec sa
Interface: FastEthernet0/1
Crypto map tag: Tar_Map_VPN, local addr 10.0.23.1

protected vrf: (none)
local ident (addr/mask/prot/port): (10.0.22.0/255.255.255.0/0/0)
remote ident (addr/mask/prot/port): (10.0.11.0/255.255.255.0/0/0)
current_peer 10.0.12.1 port 500
  PERMIT, flags={origin_is_acl,}
#pkts encaps: 3, #pkts encrypt: 3, #pkts digest: 3
#pkts decaps: 4, #pkts decrypt: 4, #pkts verify: 4
#pkts compressed: 0, #pkts decompressed: 0
#pkts not compressed: 0, #pkts compr. failed: 0
#pkts not decompressed: 0, #pkts decompress failed: 0
#send errors 0, #recv errors 0

local crypto endpt.: 10.0.23.1, remote crypto endpt.: 10.0.12.1
path mtu 1500, ip mtu 1500, ip mtu idb FastEthernet0/1
current outbound spi: 0x808C1153(2156663123)

Inbound esp sas:
  spi: 0xBEBBCD21(3199978785)
  transform: esp-aes esp-sha-hmac ,
  in use settings ={Tunnel,}
  conn id: 1, flow_id: SW-1, crypto map: Tar_Map_VPN
  sa timing: remaining key lifetime (k/sec): (4521784/3553)
  IV size: 16 bytes
  replay detection support: Y
  Status: ACTIVE

Inbound ah sas:

Inbound pcp sas:

outbound esp sas:
  spi: 0x808C1153(2156663123)
  transform: esp-aes esp-sha-hmac ,
  in use settings ={Tunnel,}
  conn id: 2, flow_id: SW-2, crypto map: Tar_Map_VPN
  sa timing: remaining key lifetime (k/sec): (4521784/3548)
  IV size: 16 bytes
  replay detection support: Y
  Status: ACTIVE

outbound ah sas:

outbound pcp sas:
```

Fig. 17. IPsec SA on GwSiteB - matching SAs with swapped SPI values

The SPI values were consistent across both gateways: the outbound SPI on GwSiteA (0x98099E7) matched the inbound SPI on GwSiteB, and vice versa (0x56AD96A1), which is the expected behaviour for a correctly established bidirectional IPsec tunnel.

2) *Remote Access Topology - VPNC*: The ISAKMP SA output on GwCorp confirmed that IKE Phase 1 completed

successfully with RemotePC (10.0.23.1), showing state QM_IDLE with conn-id 1002 and status ACTIVE.

```
GwCorp#show crypto isakmp sa
IPv4 Crypto ISAKMP SA
dst          src          state          conn-id status
10.0.12.1    10.0.23.1    QM_IDLE       1001 ACTIVE
```

Fig. 18. ISAKMP SA on GwCorp - Phase 1 established with RemotePC

The show crypto ipsec sa output on GwCorp revealed several differences relative to the site-to-site case, reflecting the dynamic nature of the remote access configuration. The crypto map tag was TAR_VPN_MAP and the local address was 10.0.12.1. The remote identity showed 10.0.100.2/255.255.255.255, corresponding to the virtual IP address assigned to RemotePC from the TAR_VPN_POOL pool, rather than its physical interface address. The local identity was 0.0.0.0/0.0.0.0, consistent with the wildcard peer configured on the server side.

The active ESP SAs used the esp-des esp-sha-hmac transform set, corresponding to transform set TAR_TS2, in tunnel mode, with status ACTIVE. The IV size of 8 bytes is consistent with DES, confirming that the negotiation fell back to the 3DES/DES-compatible policy rather than AES-256, likely due to the Enable weak encryption directive required by the vpnc client. Packet counters showed 4 encapsulated and 4 decapsulated packets, confirming bidirectional encrypted communication between RemotePC and the corporate network.

```
GwCorp#show crypto ipsec sa
Interface: FastEthernet0/1
Crypto map tag: TAR_VPN_MAP, local addr 10.0.12.1

protected vrf: (none)
local ident (addr/mask/prot/port): (0.0.0.0/0.0.0.0/0/0)
remote ident (addr/mask/prot/port): (10.0.100.1/255.255.255.0/0)
current_peer 10.0.23.1 port 500
PERMIT, flags={
  #pkts encaps: 0, #pkts encrypt: 0, #pkts digest: 0
  #pkts decaps: 0, #pkts decrypt: 0, #pkts verify: 0
  #pkts compressed: 0, #pkts decompressed: 0
  #pkts not compressed: 0, #pkts compr. failed: 0
  #pkts not decompressed: 0, #pkts decompress failed: 0
  #send errors 0, #recv errors 0

  local crypto endpt.: 10.0.12.1, remote crypto endpt.: 10.0.23.1
  path mtu 1500, ip mtu 1500, ip mtu idb FastEthernet0/1
  current outbound spi: 0xA0B4EFCF(2914316239)
  PFS (Y/N): N, DH group: none

  inbound esp sas:
    spi: 0x5099C203(1570357763)
    transform: esp-des esp-sha-hmac ,
    in use settings = (Tunnel, )
    conn id: 1, flow_id: 1, sibling_flags 00000040, crypto map: TAR_VPN_MAP
    sa timing: remaining key lifetime (k/sec): (4600000/3587)
    IV size: 8 bytes
    replay detection support: Y
    Status: ACTIVE(ACTIVE)

  inbound ah sas:

  inbound pcp sas:

  outbound esp sas:
    spi: 0xA0B4EFCF(2914316239)
    transform: esp-des esp-sha-hmac ,
    in use settings = (Tunnel, )
    conn id: 2, flow_id: 2, sibling_flags 00000040, crypto map: TAR_VPN_MAP
    sa timing: remaining key lifetime (k/sec): (4600000/3587)
    IV size: 8 bytes
    replay detection support: Y
    Status: ACTIVE(ACTIVE)

  outbound ah sas:

  outbound pcp sas:
```

Fig. 19. IPsec SA on GwCorp - DES tunnel active with RemotePC virtual IP

3) *Remote Access Topology - StrongSwan*: The StrongSwan tunnel was validated on the server using the swanctl monitoring command:

```
sudo swanctl --list-sas
```

The output confirmed that the IKEv2 Security Association was successfully established. The connection remote-access #2 showed state ESTABLISHED, using IKEv2 with AES-256-CBC encryption and SHA-256 integrity (HMAC_SHA2_256_128), with DH group MODP_2048. The remote identity was confirmed as tarteste via EAP authentication, and the virtual IP address 10.0.100.1/32 was correctly assigned to RemotePC from the server pool.

The child SA remote-access #2 was installed in tunnel mode with ESP using AES-256-CBC and SHA-256, protecting traffic between the server's internal network (10.0.11.0/24) and the client's virtual address (10.0.100.1/32).

```
remote-access: #2, ESTABLISHED, IKEv2, 3971820f580833f9_1 0b3531702ab42d7a_p*
local '10.0.12.1' 0 10.0.12.1[4500]
remote '10.0.23.1' 0 10.0.23.1[4500] EAP: 'tarteste' (10.0.100.1)
AES_CBC_256/HMAC_SHA2_256_128/PRF_HMAC_SHA2_256/MODP_2048
established 10s ago, rekeying in 13274s
remote-access: #2, reqid 1, INSTALLED, TUNNEL, ESP:AES_CBC-256/HMAC_SHA2_256_1
28
installed 10s ago, rekeying in 3274s, expires in 3942s
in c5e671c8, 0 bytes, 0 packets
out c415a56b, 0 bytes, 0 packets
local 10.0.11.0/24
remote 10.0.100.1/32
tar@localhost ~$
```

Fig. 20. StrongSwan SA output confirming IKEv2 tunnel establishment

VI. EXPERIMENTAL RESULTS

This section presents the experimental results obtained during the evaluation of the implemented IPsec VPN solution. Measurements were conducted for both site-to-site and remote access scenarios with and without IPsec enabled.

A. Throughput

Throughput was measured using iperf3, generating both TCP and UDP traffic streams over a 10-second interval between PCSiteA (10.0.11.2) and PCSiteB (10.0.22.2). For each protocol, the sender bitrate (reported by PCSiteA) and the receiver bitrate (reported by PCSiteB) were recorded. UDP tests were run with a target bitrate of 1 Mbits/s, and jitter at the receiver was also recorded.

TABLE VII
TCP THROUGHPUT - WITH AND WITHOUT IPSEC

Test	With IPsec		Without IPsec	
	Sender (Mbits/s)	Receiver (Mbits/s)	Sender (Mbits/s)	Receiver (Mbits/s)
1	1.57	1.25	2.62	0.877
2	1.57	1.25	2.31	0.964
3	1.57	1.25	2.62	0.965
4	1.57	1.25	2.52	0.965
5	1.57	1.25	2.41	0.891
Mean	1.57	1.25	2.496	0.932

1) *TCP Throughput*: The results show that the sender throughput was approximately 37% lower with IPsec active (1.57 Mbits/s) compared to the baseline without IPsec (2.496

Mbits/s). This reduction is consistent with the computational overhead introduced by the AES-128 encryption and SHA-1 integrity verification performed on each packet by the gateway routers.

The lower receiver throughput observed in the baseline (no IPsec) tests (0.932 Mbits/s) compared to the IPsec-enabled tests (1.25 Mbits/s) is attributed to the virtualized environment rather than to IPsec itself. GNS3 relies on software-based emulation, where all virtual devices share a single host CPU, introducing non-deterministic scheduling delays and variable congestion. In the baseline scenario, the gateway routers perform simple forwarding and compete more aggressively for CPU time during high-rate TCP transfers, which can lead to retransmissions and reduced receiver throughput. By contrast, with IPsec enabled, the additional cryptographic processing naturally paces traffic bursts, reducing congestion artifacts caused by the emulator. The greater variability observed in the sender throughput of the no-IPsec tests (2.31-2.62 Mbits/s) further supports the conclusion that these differences result from the virtualization layer rather than from any throughput improvement introduced by IPsec.

TABLE VIII
UDP THROUGHPUT AND JITTER - WITH AND WITHOUT IPSEC

Test	With IPsec			Without IPsec		
	Sender (Mbits/s)	Receiver (Mbits/s)	Jitter (ms)	Sender (Mbits/s)	Receiver (Mbits/s)	Jitter (ms)
1	1.05	1.04	0.764	1.05	1.04	0.623
2	1.05	1.04	0.765	1.05	1.04	0.581
3	1.05	1.04	0.594	1.05	1.04	0.701
4	1.05	1.04	0.682	1.05	1.04	0.716
5	1.05	1.04	0.755	1.05	1.04	0.770
Mean	1.05	1.04	0.712	1.05	1.04	0.678

2) *UDP Throughput*: UDP throughput was identical in both conditions (1.05 Mbits/s sender, 1.04 Mbits/s receiver), with zero packet loss in all tests. This is expected behavior: unlike TCP, UDP does not adapt its transmission rate in response to network conditions, so the throughput remained at the rate set by `iperf3` regardless of IPsec being active.

Regarding jitter, the mean value with IPsec (0.712 ms) was slightly higher than without IPsec (0.678 ms), which is consistent with the expected behavior, the additional processing required for encryption and integrity verification at the gateway introduces a small degree of variability in inter-packet arrival times. However, the difference is minimal (0.034 ms), suggesting that in this virtualized environment the dominant source of jitter is the GNS3 platform itself rather than the IPsec processing overhead.

B. Tunnel Establishment Time

To evaluate the time required to establish the IPsec tunnel an ICMP echo request sequence was initiated from `PCSiteA` to `textttPCSiteB`. Packets transmitted during the initialization period were discarded by the router, as the tunnel was not yet active to carry encrypted traffic. The number of lost packets and the total tunnel establishment time were recorded across five independent tests.

1) *Packet Loss During Negotiation*: All five tests resulted in exactly 34 lost ICMP packets during IKE negotiation. Given that each ICMP echo request is sent at a 1-second interval, this corresponds to approximately 34 seconds of traffic unavailability. The consistent result suggests deterministic behavior in how the gateways handle tunnel establishment in this environment.

2) *IKE Negotiation Duration and Retransmission Overhead*: The tunnel establishment time was measured using Wireshark packet captures on the ISP interface, recording the timestamp of the first ISAKMP message (start of IKE Phase 1) and the timestamp of the first ESP packet (indicating an active tunnel).

TABLE IX
IKE TUNNEL ESTABLISHMENT TIME (COLD START)

Test	1st ISAKMP (s)	1st ESP (s)	Duration (s)
1	35.439	52.694	17.256
2	42.640	59.554	16.914
3	42.285	59.175	16.890
4	56.307	73.251	16.944
5	5.917	23.773	17.856
Mean	36.517	53.689	17.172

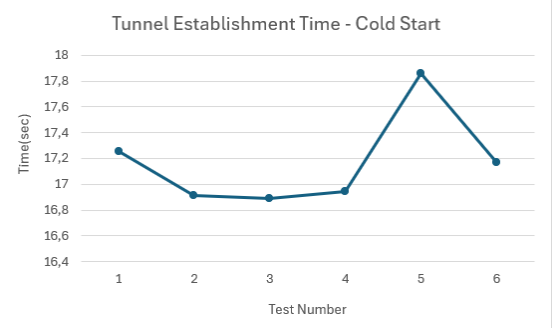


Fig. 21. Tunnel establishment time (Cold Start)

The mean observed establishment time was 17.172 seconds, with low variability across tests (range: 16.890-17.856 seconds). However, packet-level analysis revealed that this duration is heavily dominated by an initial retransmission timeout rather than active cryptographic processing.

During a "cold start", the initial ISAKMP Main Mode packet sent by the initiator is typically ignored by the remote peer (likely due to cryptographic engine wake-up delays or initial ARP resolution). Consequently, the initiating gateway waits for its retransmission timer to expire (approximately 16.5 seconds) before sending a second request.

Once the remote peer responds to the retransmitted packet, the actual active negotiation is remarkably fast. The WireShark capture confirms the standard two-phase structure: IKE Phase 1 (Main Mode, 6 messages) followed by IKE Phase 2 (Quick Mode, 3 messages). This active exchange completes in fractions of a second, after which ESP traffic begins flowing immediately.

23 71.984415	10.0.12.1	10.0.23.1	ISAKMP	210 Identity Protection (Main Mode)
24 80.003254	ca:03:0c:88:00:00	ca:03:0c:88:00:00	LOOP	60 Reply
25 94.004066	c0:04:11:0f:00:01	c0:04:11:0f:00:01	LOOP	60 Reply
26 88.811081	10.0.12.1	10.0.23.1	ISAKMP	210 Identity Protection (Main Mode)
27 88.872040	10.0.12.1	10.0.23.1	ISAKMP	150 Identity Protection (Main Mode)
28 88.892163	10.0.12.1	10.0.23.1	ISAKMP	346 Identity Protection (Main Mode)
29 88.902877	10.0.12.1	10.0.23.1	ISAKMP	346 Identity Protection (Main Mode)
30 89.012353	10.0.12.1	10.0.23.1	ISAKMP	350 Identity Protection (Main Mode)
31 89.053731	10.0.12.1	10.0.23.1	ISAKMP	118 Identity Protection (Main Mode)
32 89.074137	10.0.12.1	10.0.23.1	ISAKMP	230 Quick Mode
33 89.114500	10.0.12.1	10.0.23.1	ISAKMP	230 Quick Mode
34 89.134660	10.0.12.1	10.0.23.1	ISAKMP	102 Quick Mode
35 89.840739	10.0.12.1	10.0.23.1	ESP	186 ESP (SPI=0xb3321a)

Fig. 22. Wireshark capture showing IKE Phase 1 (ISAKMP), Phase 2 (Quick Mode) and first ESP packets on the ISP interface

Finally, the 34-second packet loss period significantly exceeds the approximately 17-seconds IKE establishment process. This discrepancy occurs because ICMP packets generated before the IKE process is fully triggered (i.e., before the first ISAKMP message is dispatched) are also dropped by the router due to the absence of an active SA.

3) *Warm Start Tunnel Establishment*: To isolate the actual cryptographic and protocol processing overhead from the initial network and hardware initialization delays, a subsequent "warm start" test was conducted. In this scenario, the active Security Associations were cleared, but the gateway's cryptographic engine remained active, and environmental state variables (such as ARP caches) were already populated.

Under these conditions, when interesting traffic was generated to trigger the VPN, the tunnel establishment was nearly instantaneous. The entire IKE negotiation-encompassing both Phase 1 (Main Mode) and Phase 2 (Quick Mode)-completed in approximately 0.25 seconds. This drastic reduction empirically confirms that the ~17-seconds delay observed during the cold start is entirely an artifact of initial environmental timeouts and retransmissions, rather than the intrinsic computational or architectural overhead of the IPsec/IKEv1 implementation.

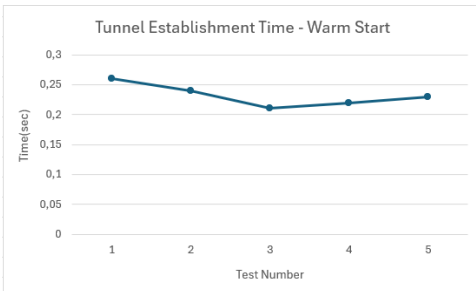


Fig. 23. Tunnel establishment times showing the active IKE negotiation

C. ESP Packet Overhead

To quantify the overhead introduced by IPsec encapsulation, Wireshark packet captures were performed on the link between GwSiteA and the ISP router, comparing the total frame size of ICMP packets with and without IPsec active. A standard ping with the default payload size (56 bytes) was used as the reference traffic.

TABLE X
FRAME SIZE COMPARISON - WITH AND WITHOUT IPSEC

Condition	Frame Size (bytes)
Without IPsec (plain ICMP)	98
With IPsec (ESP encapsulated)	166
ESP Overhead	68

The 68-byte overhead remains consistent regardless of the payload size. This was confirmed by the 1000-byte payload test, which revealed a difference of exactly 68 bytes. The result was 1042 bytes without IPsec versus 1110 bytes with IPsec. This demonstrates that the ESP overhead remains constant regardless of the payload size. It can be broken down as follows:

TABLE XI
DECOMPOSITION OF THE 68-BYTE ESP OVERHEAD

Field	Size (bytes)
Outer IP header	20
ESP header (SPI + Sequence Number)	8
IV (AES-128)	16
ESP trailer (padding + pad length + next header)	~12
ICV (SHA-1 HMAC)	12
Total	~68

The captures also confirm the encapsulation structure: without IPsec, the Wireshark dissector shows a plain ICMP packet with source and destination addresses corresponding to the end hosts (10.0.11.2 and 10.0.22.2). With IPsec active, the same traffic appears as an ESP packet whose outer IP header carries the gateway addresses (10.0.12.1 and 10.0.23.1), completely concealing the original packet from intermediate network devices.

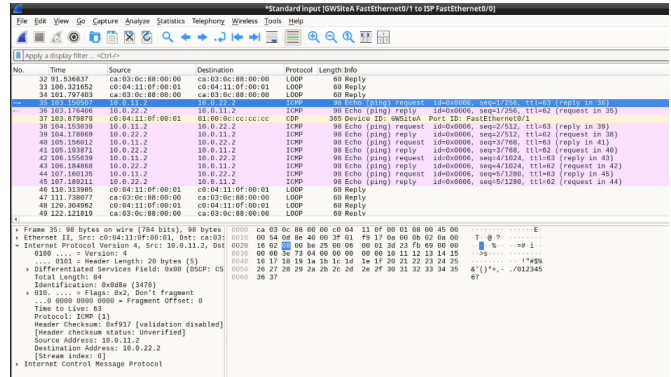


Fig. 24. Wireshark capture without IPsec - plain ICMP frame, 98 bytes

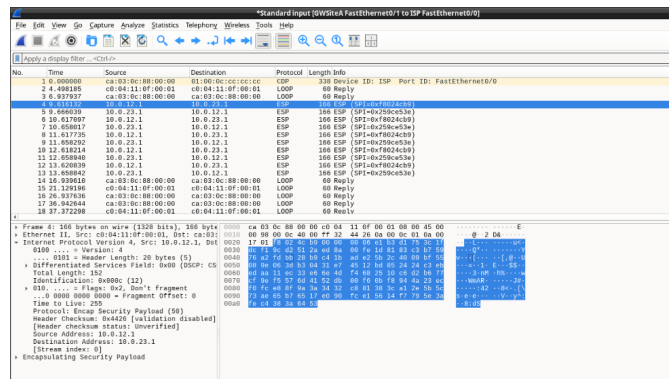


Fig. 25. Wireshark capture with IPsec - ESP encapsulated frame, 166 bytes

D. RTT Analysis by Packet Size

To evaluate how the IPsec overhead affects latency as a function of packet size, RTT was measured using ICMP echo requests of four different payload sizes: 100, 500, 1000, and 1400 bytes. Five measurements were performed for each size, with and without IPsec active, and the mean RTT was recorded.

TABLE XII
MEAN RTT BY PACKET SIZE - WITH AND WITHOUT IPSEC

Size (bytes)	RTT without IPsec (ms)	RTT with IPsec (ms)	Difference (ms)	Overhead (%)
100	37.761	43.618	+5.857	+15.5%
500	41.810	47.573	+5.763	+13.8%
1000	40.105	48.559	+8.454	+21.1%
1400	41.417	68.891	+27.474	+66.3%

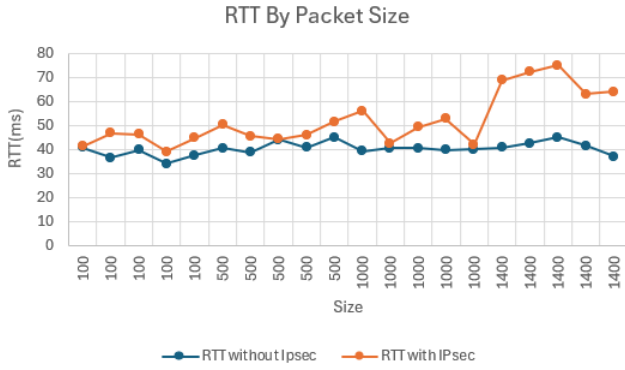


Fig. 26. RTT by Packet Size - With and Without IPsec

For packet sizes up to 1000 bytes, the RTT overhead introduced by IPsec was relatively modest and consistent, ranging from approximately 5.7 ms to 8.5 ms. This represents the fixed cost of AES-128 encryption and SHA-1 integrity verification performed by the gateway routers on each packet.

However, a significant increase was observed at 1400 bytes, where the RTT with IPsec (68.891 ms) was 66.3% higher than without IPsec (41.417 ms), an additional latency of approximately 27 ms compared to the ~6 ms overhead seen at smaller sizes. This abrupt increase is directly attributable to packet fragmentation, as discussed in the following subsection.

E. Packet Fragmentation

The IPsec encapsulation process adds approximately 68 bytes of overhead to each packet. When the original payload is large enough, the resulting ESP frame may exceed the MTU of 1500 bytes, forcing the gateway to fragment the packet before transmission.

For a packet with a 1400-byte ICMP payload, the total size without IPsec is:

$$1400 + 20_{IP} + 8_{ICMP} = 1428 \text{ bytes} < 1500 \text{ bytes (MTU)} \quad (1)$$

With IPsec encapsulation, the same packet becomes:

$$1428 + 68_{ESP \text{ overhead}} \approx 1496 \text{ bytes} \lesssim 1500 \text{ bytes (MTU)} \quad (2)$$

At 1450 bytes of payload, the total ESP frame size exceeds the MTU:

$$1450 + 20 + 8 + 68 = 1546 \text{ bytes} > 1500 \text{ bytes (MTU)} \quad (3)$$

This was confirmed experimentally by sending a ping with a 1450-byte payload and the Don't Fragment (DF) bit disabled (-M dont) to force fragmentation:

```
ping -c 5 -s 1450 -M dont 10.0.22.2
```

Analysis of the Wireshark capture on the ISP link revealed that the original ESP frame had split into two fragments: one containing 1494 bytes, and the other containing 150 bytes. This confirmed that fragmentation had occurred at the gateway level. Without IPsec, the 1450-byte payload (with a total frame size of 1478 bytes) would have been transmitted as a packet, as it would have remained within the MTU limit. Therefore, fragmentation would not have been required.

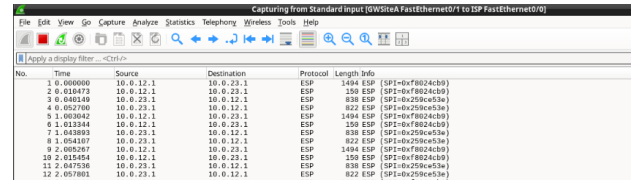


Fig. 27. Wireshark capture showing ESP fragmentation

The impact of fragmentation on latency is clearly visible in Table XII: the RTT at 1400 bytes with IPsec is 66.3% higher than without IPsec, compared to only 13-21% overhead at smaller sizes.

Because this topology implements pre-encapsulation fragmentation, the disproportionate RTT increase is not caused by network-level IP reassembly dependencies at the destination. Instead, the latency penalty stems from architectural overhead on the gateways: the initiating router must split the cleartext packet and generate two distinct, fully-formed ESP frames, doubling the cryptographic workload. Furthermore, handling fragmented packets frequently breaks the router's hardware-accelerated forwarding path (ASIC), forcing the device to process these frames via the main CPU (*process switching*). This creates internal queuing delays that manifest as a substantial RTT amplification.

When the DF bit is set (default ping behavior), the gateway cannot fragment the packet and instead discards it, sending an ICMP "Fragmentation Needed" error back to the source with the effective MTU value (1427 bytes in this experiment). This behavior was observed when running the ping without the -M dont flag, which resulted in 20% packet loss and the error message:

```
From 10.0.11.2 icmp_seq=1
Frag needed and DF set (mtu = 1427)
```

This MTU value of 1427 bytes represents the maximum payload size that can be carried through the IPsec tunnel without fragmentation, accounting for the 68-byte ESP overhead within the 1500-byte Ethernet MTU.

F. Remote - VPNC

The remote access topology presented distinct behavior across all evaluated metrics when compared to the site-to-site scenario, reflecting the fundamental architectural differences between the two deployment models.

1) **Tunnel Establishment:** During the remote access evaluation, the tunnel was consistently established in under one second. Unlike the initial site-to-site cold start-which suffered a ~17-second delay strictly due to environmental timeouts-the remote access client avoided these initial packet drops and connected reliably.

When comparing the protocol mechanics directly against the site-to-site warm start (which completed in ~0.25 seconds), the remote access establishment presents a structural trade-off. To accelerate the initial connection, the remote access client (vpnc) utilized IKEv1 Aggressive Mode, which condenses Phase 1 into just three messages. In contrast, the site-to-site topology utilized the standard IKEv1 Main Mode, requiring six messages.

However, the theoretical speed advantage of Aggressive Mode is entirely offset by the architectural requirement of Extended Authentication (XAUTH). Because it operates on a client-to-gateway model, the remote access tunnel mandates user-level credential validation (Config Mode) prior to Phase 2. This introduces an additional exchange sequence that ultimately makes the overall negotiation slightly heavier and more time-consuming than the pure machine-to-machine site-to-site process.

The Wireshark capture on the ISP link confirmed this behavior, showing the complete remote access negotiation sequence:

- 1) **Aggressive Mode (3 messages):** IKE Phase 1, combining the SA proposal, Diffie-Hellman exchange, and peer identity into a single exchange
- 2) **Config Mode / XAUTH (6 messages):** extended authentication, where the gateway verified the user credentials (tarteste) configured in the vpnc client
- 3) **Quick Mode (3 messages):** IKE Phase 2, negotiating the IPsec Security Associations
- 4) **ESP packets:** encrypted tunnel traffic begins immediately after negotiation completes

No.	Time	Source	Destination	Protocol	Length	Info
21	47.618432	10.0.2.1	10.0.12.1	ISAKMP	1551	Aggressive
22	47.792803	10.0.12.1	10.0.2.1	ISAKMP	408	Aggressive
23	47.789803	10.0.2.1	10.0.12.1	ISAKMP	214	Aggressive
24	47.848099	10.0.12.1	10.0.2.1	ISAKMP	142	Informational
25	47.840208	10.0.12.1	10.0.2.1	ISAKMP	138	Transaction (Config Mode)
26	47.860402	10.0.2.1	10.0.12.1	ISAKMP	130	Transaction (Config Mode)
27	47.870708	10.0.12.1	10.0.2.1	ISAKMP	130	Transaction (Config Mode)
28	47.891129	10.0.2.1	10.0.12.1	ISAKMP	130	Transaction (Config Mode)
29	47.901231	10.0.2.1	10.0.12.1	ISAKMP	230	Transaction (Config Mode)
30	47.911101	10.0.12.1	10.0.2.1	ISAKMP	390	Transaction (Config Mode)
31	48.003303	10.0.2.1	10.0.12.1	ISAKMP	602	Quick Mode
32	48.133961	10.0.12.1	10.0.2.1	ISAKMP	246	Quick Mode
33	48.156451	10.0.2.1	10.0.12.1	ISAKMP	84	Quick Mode
34	48.156429	10.0.2.1	10.0.12.1	ISAKMP	128	Informational
35	48.156445	10.0.2.1	10.0.12.1	ESP	138	ESP (SPI=0x20c4110)
36	48.164538	10.0.12.1	10.0.2.1	ISAKMP	134	Informational
37	48.308529	10.0.2.1	10.0.12.1	ESP	138	ESP (SPI=0x20c4110)

Fig. 28. Wireshark capture showing differences in tunnel establishment

While Aggressive Mode requires fewer initial exchanges, it offers a reduced security margin compared to Main Mode: peer identities are transmitted in cleartext before encryption is fully established, making them potentially susceptible to offline dictionary attacks. This trade-off between speed and security, combined with the mandatory XAUTH layer for user validation, is a defining characteristic of legacy remote access VPN deployments.

2) **ESP Packet Overhead:** Wireshark captures on the ISP link were used to compare the frame size of ICMP packets with and without the IPsec tunnel active, using the same methodology applied in the site-to-site evaluation.

TABLE XIII
FRAME SIZE COMPARISON - REMOTE ACCESS (56-BYTE ICMP PAYLOAD)

Condition	Frame Size (bytes)
Without IPsec (plain ICMP)	98
With IPsec (ESP encapsulated)	150
ESP Overhead	52

The remote access topology introduced an ESP overhead of 52 bytes per packet, compared to 68 bytes in the site-to-site scenario, a difference of 16 bytes. This reduction results from two factors related to the DES algorithm negotiated by the vpnc client (transform set TAR_TS2: esp-des esp-sha-hmac). First, DES uses an 8-byte initialization vector, compared to the 16-byte IV of AES-128, accounting for 8 bytes of the difference. Second, DES operates on 8-byte blocks rather than the 16-byte blocks of AES, which reduces the average padding required in the ESP trailer, accounting for the remaining difference. The outer IP header, ESP header, and ICV fields are identical between the two topologies.

TABLE XIV
ESP OVERHEAD COMPARISON - SITE-TO-SITE VS REMOTE ACCESS

Condition	Site-to-Site	Remote Access
Without IPsec	98 bytes	98 bytes
With IPsec	166 bytes	150 bytes
ESP Overhead	68 bytes	52 bytes

3) **Packet Fragmentation:** The remote access topology exhibited a fundamentally different architecture for triggering fragmentation when compared to the other deployment models, primarily driven by the explicit MTU configuration of the virtual tunnel interface created by vpnc.

Upon establishing the tunnel, vpnc automatically configured the tun0 interface with a reduced MTU of 1412 bytes. This drop from the standard 1500-byte Ethernet MTU is deliberately engineered to account for the maximum anticipated ESP encapsulation overhead. Consequently, the Linux kernel fragments large packets at the cleartext IP level *before* handing them over to the IPsec process. This behavior represents a textbook implementation of a **Pre-Encapsulation Fragmentation** model.

```
ip link show tun0
tun0: mtu 1412
```

For an ICMP payload of 1400 bytes, fragmentation was confirmed by the Wireshark capture on the ISP link. This showed that the original packet was divided into two independent ESP-encapsulated fragments prior to transmission:

TABLE XV
FRAGMENT SIZES - REMOTE ACCESS VS SITE-TO-SITE (1400-BYTE PAYLOAD)

	Site-to-Site	Remote Access
Fragment 1	1494 bytes (ESP)	1478 bytes (ESP)
Fragment 2	150 bytes (ESP)	102 bytes (ESP)

While this pre-encapsulation approach introduces additional packets into the network, it provides a substantial operational advantage over post-encapsulation models. Because each packet is encrypted independently as a complete ESP frame, the receiving gateway does not encounter network-level IP reassembly dependencies before decryption can begin; each incoming frame can be processed immediately upon arrival.

Furthermore, unlike the site-to-site scenario where generating double the amount of ESP frames breaks hardware-accelerated ASICs and causes a severe RTT penalty due to CPU process switching on the routers—the *vpnc* client executes this workload entirely within the host’s high-performance general-purpose CPU. As a result, the overhead of managing these independent cryptographic frames is easily absorbed by the computer’s system resources, maintaining low and stable latency even at maximum packet sizes.

G. Remote Access - StrongSwan

The StrongSwan-based remote access topology was evaluated using the same methodology applied to the *vpnc* scenario, allowing a direct comparison between the two remote access implementations.

1) *Tunnel Establishment*: The StrongSwan tunnel was established in approximately 0.25 seconds. While this is drastically faster than the initial site-to-site cold start (~17 seconds due to environmental timeouts), it performs identically to the site-to-site warm start (~0.25 seconds) and remains comparable to the *vpnc* implementation (<1 second).

The negotiation used IKEv2, which differs structurally from the IKEv1 modes used in the other two scenarios. The Wireshark capture on the ISP link confirmed the following sequence:

- 1) **IKE_SA_INIT (2 messages)**: establishes the cryptographic channel, exchanging cipher proposals and Diffie-Hellman values in a single round-trip
- 2) **IKE_AUTH (multiple exchanges)**: authenticates the server using its PKI certificate and the client using EAP-MSCHAPv2 credentials, with IKEv2 message fragmentation observed (fragment 1/2, fragment 2/2) due to the size of the certificate payload
- 3) **ESP packets**: encrypted tunnel traffic begins immediately after negotiation completes

Fig. 29. Wireshark capture showing IKEv2 IKE_SA_INIT and IKE_AUTH exchanges during StrongSwan tunnel establishment

Unlike *vpnc*, which uses IKEv1 Aggressive Mode and reveals the peer’s identity before encryption is established, IKEv2 keeps all identity information secure within the encrypted IKE_AUTH exchange. This provides stronger security guarantees during negotiation. Furthermore, server authentication relies on a PKI certificate rather than a pre-shared key, thus eliminating the risks associated with static shared secrets.

2) *ESP Packet Overhead*: The same methodology used for previous evaluations was employed to compare the frame sizes of ICMP packets with and without an active StrongSwan tunnel. This was done using Wireshark captures on the ISP link.

TABLE XVI
FRAME SIZE COMPARISON - STRONGSWAN REMOTE ACCESS (56-BYTE ICMP PAYLOAD)

Condition	Frame Size (bytes)
Without IPsec (plain ICMP)	98
With IPsec (ESP encapsulated)	170
ESP Overhead	72

The StrongSwan topology introduced an ESP overhead of 72 bytes per packet, the largest of the three scenarios evaluated. This is a direct consequence of the stronger cryptographic algorithms negotiated during IKEv2: AES-256 encryption and SHA-256 integrity (HMAC_SHA2_256_128). The SHA-256 HMAC produces a 16-byte ICV, compared to the 12-byte ICV of SHA-1 used in the site-to-site and *vpnc* configurations, accounting for the 4-byte difference relative to the site-to-site overhead.

TABLE XVII
ESP OVERHEAD COMPARISON - ALL SCENARIOS

Condition	Site-to-Site	vpnc	StrongSwan
Without IPsec	98 bytes	98 bytes	98 bytes
With IPsec	166 bytes	150 bytes	170 bytes
Overhead	68 bytes	52 bytes	72 bytes
Encryption	AES-128	DES	AES-256
Integrity	SHA-1	SHA-1	SHA-256

3) *Packet Fragmentation*: Unlike *vpnc*, which creates a *tun0* virtual interface with an MTU of 1412 bytes, StrongSwan uses the Linux kernel’s XFRM subsystem to apply IPsec transformations inline on the *ens3* physical interface. This retains the standard MTU of 1,500 bytes for the interface. Consequently, StrongSwan’s fragmentation behaviour differs fundamentally from that of *vpnc*.

```
ip link show ens3
ens3: mtu 1500
```


Since the kernel does not artificially reduce the MTU prior to encryption, it constructs the full, oversized ESP packet first, and only then applies IP-level fragmentation to fit the 1 500-byte physical limit. This execution order represents a standard Post-Encapsulation Fragmentation model. This was confirmed by the Wireshark capture, which displayed packets explicitly flagged as IPv4 Fragmented IP protocol (proto=Encap Security Payload) with the *More Fragments* flag actively set.

TABLE XVIII
FRAGMENT SIZES - ALL SCENARIOS (1450-BYTE PAYLOAD)

	Site-to-Site	vpnc	StrongSwan
Fragment 1	1 494 bytes (ESP)	1 478 bytes (ESP)	1 514 bytes (IPv4/ESP)
Fragment 2	150 bytes (ESP)	102 bytes (ESP)	146 bytes (ESP)
More Fragments flag	Not set	Not set	Set

The presence of the *More Fragments* flag exclusively in the StrongSwan implementation highlights a critical architectural divergence. While the site-to-site and vpnc topologies perform pre-encapsulation fragmentation-generating independent, fully-formed ESP packets that can be decrypted immediately upon arrival-StrongSwan forces the receiving gateway to perform network-level IP reassembly **before** any decryption can take place. If any single IP fragment is dropped or delayed in transit, the entire ESP payload cannot be reconstructed, leading to potential latency amplification and higher CPU overhead under heavy fragmentation.

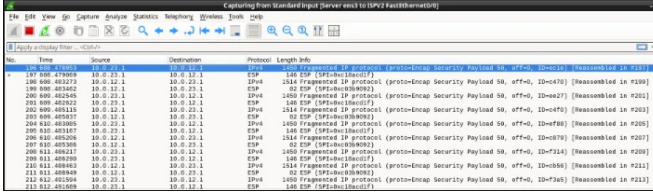


Fig. 30. Wireshark capture showing IPv4 fragmentation with More Fragments flag set during StrongSwan tunnel transmission

VII. DISCUSSION

The experimental results provide a comprehensive overview of the trade-offs involved in deploying an IPsec-based VPN. This section discusses the security benefits demonstrated by the implementation, the performance costs observed during testing and the limitations of the virtualized experimental environment.

A. Site to Site Topology

1) *Security Benefits*: The implementation successfully demonstrated the core security properties of IPsec. The Wireshark captures provided the most direct evidence of the confidentiality guarantee: without IPsec, traffic between PCSiteA and PCSiteB was fully visible on the ISP link, with source and destination addresses, protocol information, and payload all readable by any intermediate device. With IPsec active, the same traffic appeared exclusively as ESP packets carrying the gateway addresses, with the original packet completely

concealed inside the encrypted payload. An attacker positioned on the public network would be unable to determine the identity of the communicating hosts, the protocol in use, or the content of the exchanged data.

Data integrity and authentication were ensured through the ICV field included in each ESP packet, computed using SHA-1 HMAC with the shared session key. The correct matching of SPI values across both gateways, confirmed by the `show crypto ipsec sa` output, verified that both endpoints were operating under the same Security Association and that packets were being correctly authenticated before delivery.

One particularly relevant property for practical deployment is the solution's transparency to end hosts. PCSiteA and PCSiteB did not require any VPN-specific configuration since the gateway routers handled the encryption and encapsulation entirely. This enables IPsec to be utilized in site-to-site scenarios without the necessity of modifying existing end-user devices or applications.

2) *Performance Costs*: The security guarantees provided by IPsec led to a measurable decrease in performance across all the evaluated metrics.

ESP encapsulation adds a fixed overhead of 68 bytes to every packet, regardless of the size of the payload. This reduces the effective bandwidth available for application data and, more critically, lowers the practical MTU of the tunnel to approximately 1 427 bytes. Any packet whose total size after ESP encapsulation exceeds the 1 500-byte Ethernet MTU must be fragmented. This was confirmed experimentally using Wireshark captures which showed a 1 450-byte payload being split into fragments of 1 494 and 150 bytes.

Latency measurements showed a consistent overhead of approximately 5–8 ms for packet sizes up to 1 000 bytes, representing the fixed cryptographic processing cost at the gateways. However, at 1 400 bytes, the RTT increased by 27 ms (66%) compared to the baseline. This disproportionate penalty is directly tied to the pre-encapsulation fragmentation model; splitting the traffic doubles the cryptographic workload and breaks the router's hardware-accelerated forwarding path, forcing the frames to be processed via the main CPU (*process switching*), which introduces significant queuing delays.

Finally, the initial tunnel establishment process introduced a traffic unavailability window of approximately 17 seconds, during which 34 ICMP packets were discarded. While highly consistent across tests, packet-level analysis proved that this delay is an artifact of a cold-start environment-where initial packets are dropped due to environmental timeouts and retransmission delays before the first ISAKMP message-rather than the intrinsic execution time of the IKE protocol, which completed in just 0.25 seconds under warm-start conditions.

B. Site-to-Site vs Remote Access

Although site-to-site and remote access topologies serve different purposes, with the former interconnects entire networks, while the latter provides individual user access, meaningful comparisons can still be made between them across key metrics, as demonstrated by the experimental results.

TABLE XIX
COMPARISON - SITE-TO-SITE VS REMOTE ACCESS

Metric	Site-to-Site	vpnc	StrongSwan
Tunnel establishment	~0.25s (Warm Start)	<1s (Aggressive Mode)	~0.25s (IKEv2)
ESP Overhead	68 bytes	52 bytes	72 bytes
Network hops	4	3	3
End host transparency	Yes	No	No
User authentication	None	XAUTH	EAP-MSCHAPv2

When analyzing tunnel establishment, a superficial observation might suggest that the site-to-site topology is drastically slower due to the ~17-second traffic unavailability window recorded during initial tests. However, as previously demonstrated, this delay is strictly an artifact of cold-start environmental timeouts. When isolating the pure protocol mechanics (warm start), the site-to-site IKEv1 Main Mode completes in approximately 0.25 seconds. This makes its raw cryptographic negotiation speed highly competitive with both remote access implementations, despite requiring more message exchanges.

Regarding ESP overhead, the three scenarios differ due to the cryptographic algorithms negotiated in each case. The `vpnc` client fell back to DES encryption, resulting in the smallest overhead (52 bytes), while StrongSwan negotiated AES-256 with SHA-256, producing the largest (72 bytes). The site-to-site configuration used AES-128 with SHA-1, yielding an intermediate overhead of 68 bytes. It is important to note, however, that in practice this difference in overhead has a negligible impact on network performance. Modern hardware and software implementations are highly optimized for cryptographic operations, and the additional bytes introduced by stronger algorithms represent a marginal cost compared to the bandwidth and latency characteristics of real-world networks.

From an architectural perspective, the site-to-site topology offers full transparency to end hosts: neither `PCSiteA` nor `PCSiteB` required any VPN-specific configuration, as encryption was handled entirely by the gateway routers. In contrast, both remote access implementations required VPN client software on `RemotePC`, and additionally introduced a user authentication layer (XAUTH or EAP-MSCHAPv2) absent in the site-to-site scenario. This extra authentication step represents a security advantage of the remote access model, verifying not only the device but also the identity of the user establishing the connection.

C. `vpnc` vs StrongSwan

Both `vpnc` and StrongSwan were evaluated as remote access VPN clients connecting to the corporate network. Despite serving the same functional purpose, the two implementations differ significantly in the protocols, algorithms, and security mechanisms they employ.

TABLE XX
COMPARISON - `VPNC` VS STRONGSWAN

Metric	vpnc	StrongSwan
IKE Version	IKEv1 Aggressive Mode	IKEv2
Tunnel establishment	<1 second	~0.25 seconds
ESP Overhead	52 bytes	72 bytes
Encryption Algorithm	DES	AES-256
Server Authentication	Pre-Shared Key (PSK)	PKI Certificate
User Authentication	XAUTH	EAP-MSCHAPv2

From a security standpoint, StrongSwan represents a substantial improvement over `vpnc`. The use of IKEv2 ensures that peer identities are exchanged within an already encrypted channel, unlike IKEv1 Aggressive Mode where the identity is exposed before encryption is established, making it susceptible to offline dictionary attacks against the pre-shared key. Furthermore, DES encryption, negotiated by `vpnc` due to compatibility constraints, has been cryptographically broken since 1999 and is explicitly prohibited in current security standards such as NIST SP 800-131A. By contrast, AES-256 remains secure against all known attacks. Server authentication via PKI certificates in StrongSwan also eliminates the risks associated with static pre-shared keys, which must be distributed and managed securely across all clients.

In terms of performance, StrongSwan established the tunnel slightly faster than `vpnc` (0.25s vs <1s), despite using a more complex authentication mechanism involving certificate verification and EAP-MSCHAPv2 exchanges. This efficiency is largely due to the streamlined negotiation structure of IKEv2, which reduces the number of round-trips required to establish both the IKE and IPsec Security Associations compared to IKEv1.

The larger ESP overhead of StrongSwan (72 bytes compared to 52 bytes) is a direct consequence of its use of more robust algorithms. However, as discussed in the previous section, this additional overhead has a negligible impact on practical network performance. Using AES-256 and SHA-256 does not significantly increase latency in modern environments where cryptographic acceleration is widely available. The security benefits of replacing DES with AES-256 and pre-shared keys with certificate-based authentication far outweigh the marginal increase in per-packet overhead.

Overall, StrongSwan is the superior choice for deploying remote access VPNs. It provides stronger security guarantees and a more modern protocol stack, offering comparable or better performance than the legacy `vpnc` client.

D. Limitations of the Virtualized Environment

Although GNS3 provides a flexible and realistic environment for network experimentation, virtualized infrastructures introduce limitations that may affect measurement accuracy.

The performance of virtual routers depends on the host machine's CPU and memory resources, which may introduce variability between experimental runs. Additionally, software-based forwarding and encryption do not fully replicate the behavior of dedicated hardware appliances with cryptographic acceleration.

Therefore, the obtained results should be interpreted as representative experimental observations rather than exact real-world performance values.

VIII. CONCLUSIONS

This project successfully implemented and evaluated IPsec-based VPN solutions for site-to-site and remote access deployment scenarios in a GNS3 virtualized environment. Three configurations were tested: a site-to-site tunnel using IKEv1 Main Mode with AES-128 and SHA-1, a remote access tunnel using `vpnc` with IKEv1 Aggressive Mode and DES, and a remote access tunnel using StrongSwan with IKEv2, AES-256, SHA-256, and PKI certificate authentication.

The results confirmed that IPsec effectively enforces confidentiality, integrity, authentication, and anti-replay protection, as directly evidenced by Wireshark captures showing complete traffic concealment on the public link. Performance measurements quantified the associated costs: TCP sender throughput fell by approximately 37%, latency overhead was modest at small packet sizes (5-8 ms) but rose sharply to 27 ms at 1400 bytes due to ESP-induced fragmentation.

The comparison between `vpnc` and StrongSwan revealed a significant security disparity: `vpnc` fell back to cryptographically broken DES and exposed peer identity during negotiation, whereas StrongSwan provided AES-256 encryption and full IKEv2 identity protection with comparable performance. For any new remote access deployment, a modern IKEv2-based implementation is strongly recommended.

The primary limitation of this study is the virtualized experimental environment, which introduces scheduling variability that prevents direct extrapolation to physical hardware. Future work should extend the evaluation to real devices, incorporate CPU utilization metrics, and assess alternative technologies such as WireGuard and OpenVPN.

REFERENCES

- [1] P. J. Ezra, S. Misra, A. Agrawal, J. Oluranti, R. Maskeliunas, and R. Damasevicius, "Secured communication using virtual private network (vpn)," in *Cyber Security and Digital Forensics*, K. Khanna, V. V. Estrela, and J. J. P. C. Rodrigues, Eds. Singapore: Springer Singapore, 2022, pp. 309–319.
- [2] E. Daraghmi, "Virtual private network (vpn)," 2024, zenodo preprint / technical report.
- [3] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed. Pearson, 2017, ch. 20, pp. 640–668.
- [4] S. Kent, "Ip authentication header," IETF, Tech. Rep. 4302, 2005. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc4302>
- [5] —, "Ip encapsulating security payload (esp)," IETF, Tech. Rep. 4303, 2005. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc4303>
- [6] P. Segeč, M. Moravčík, M. Kontšek, J. Papán, J. Uramová, and O. Yermenko, "Network virtualization tools – analysis and application in higher education," in *2019 17th International Conference on Emerging eLearning Technologies and Applications (ICETA)*, 2019, pp. 699–708.
- [7] R. Emiliano and M. Antunes, "Automatic network configuration in virtualized environment using gns3," in *2015 10th International Conference on Computer Science & Education (ICCSE)*, 2015, pp. 25–30.